Serverless Applications with Lambda

Caleb Quisenberry

University of Advancing Technology

June 30, 2024

Author Note

This paper was prepared for NTS336, Cloud Security, taught by Professor Aaron Jones.

Serverless Applications with Lambda

AWS Lambda is a serverless computing service provided by Amazon Web Services that allows users to run code without provisioning or managing servers. This service automatically scales applications by running code in response to triggers such as changes in data, shifts in system state, or user actions. This report will summarize the tutorial steps, explain AWS Lambda's features, discuss its applications, and highlight any challenges faced during the tutorial.

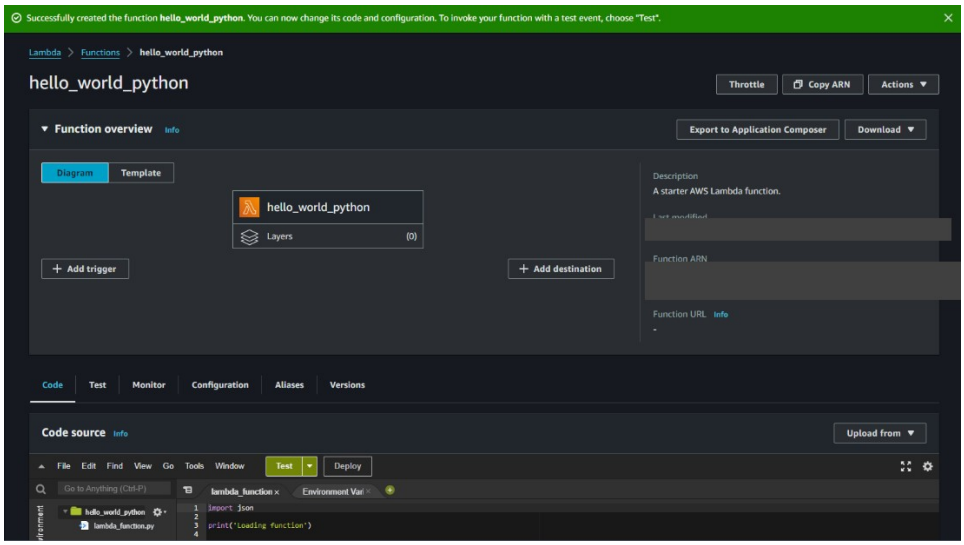
Tutorial Steps

Setting up the Lambda Function

The first step in the tutorial involved logging into the AWS Management Console and navigating to the AWS Lambda service. From there, a new Lambda function was created with the following configurations:

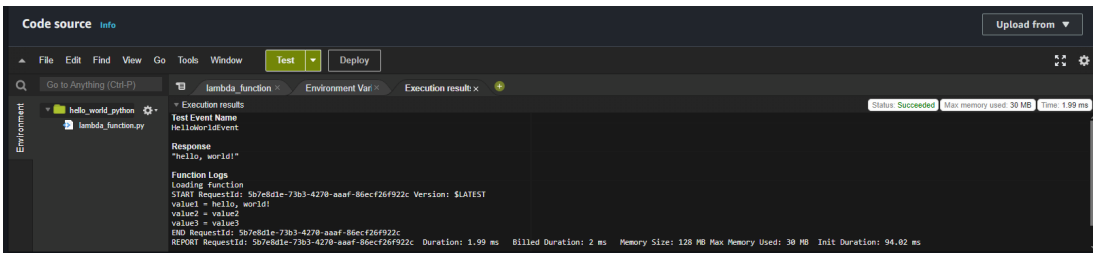
- **Function Name:** hello_world_python
- **Runtime:** Python 3.10
- **Role:** Create a new role with basic Lambda permissions
- **Choose Blueprint:** A premade code blueprint was chosen for the exercise, this can either be created by the user instead, or modified at a later step

The following screenshot shows the created function.



Creating a Test Event

To evaluate the Lambda function, a test event named HelloWorldEvent was created. This test event was then used to invoke the Lambda function, which successfully returned the expected "Hello, world!" message.



Reviewing Results

The execution results showed that the Lambda function ran successfully, with logs detailing the request ID, duration, and memory usage. This confirmed that the function was working as intended. No major obstacles were encountered during this assignment.

Key Features of AWS Lambda

AWS Lambda offers several key features:

- **Event-Driven Execution:** Automatically runs code in response to events such as HTTP requests via Amazon API Gateway, modifications to data in Amazon S3 buckets, or

updates to DynamoDB tables.

- **Scalability:** Automatically scales the execution environment by running code in parallel to oversee the volume of incoming events.
- **Pay-As-You-Go:** Charges are based only on the compute time consumed, making it cost-effective.
- **Integrated Security:** Provides built-in integration with AWS Identity and Access Management (IAM) to manage access to resources and ensure secure execution of functions.

Applications and Limitations of AWS Lambda

Beneficial Scenarios

AWS Lambda is particularly beneficial in the following scenarios:

- **Microservices Architecture:** Ideal for deploying individual microservices that can scale independently.
- **Real-Time File Processing:** Efficient for processing files in real-time, such as resizing images or transcoding videos.
- **Data Transformation:** Useful for transforming data on the fly before storing it in a database or data warehouse.
- **Event-Driven Applications:** Suitable for applications that need to respond to events, such as chatbots or IoT applications.

Limitations

Despite its advantages, AWS Lambda has some limitations:

- **Cold Starts:** Functions may experience latency during initial invocations, known as cold starts.

- **Execution Timeout:** Functions have a maximum execution timeout of 15 minutes, which may not be suitable for long-running processes.
- **Limited Control:** Less control over the underlying infrastructure compared to traditional server-based environments.

Conclusion

This tutorial provided a firsthand introduction to AWS Lambda, demonstrating how to create, deploy, and test a simple serverless function. AWS Lambda's event-driven execution, scalability, and cost-effectiveness make it a powerful tool for various applications.

References

Introduction - AWS Lambda. (n.d.). Docs.aws.amazon.com. https://docs.aws.amazon.com/lambda/latest/operatorguide/intro.html